

Introduction to Modern Neural Architectures

What are Neural Networks?

Neural Networks are a type of computer system inspired by the human brain. They consist of layers of interconnected "nodes" or neurons that process information. Instead of being given a strict set of rules by a programmer, these networks learn by looking at thousands of examples.

As data passes through each layer, the network identifies increasingly complex patterns—starting from simple edges in a picture to recognizing a full face, or from single words to the meaning of a whole sentence.

Why Sequence Models are Important

In many real-world scenarios, the order of information is just as important as the information itself. Sequence models are designed specifically for data where timing and position matter.

- **Language:** In the sentence "The bank was closed," the word "bank" means something different than in "The river bank was muddy." The model must understand the sequence of words to get the context.
- **Time-Series:** In fields like Smart Grids, predicting how much electricity a neighborhood will use requires looking at a sequence of past usage over hours or days.
- **Healthcare:** Clinical notes and patient records are sequences of events; understanding the order of symptoms is vital for an accurate diagnosis.

Limitations of RNN and LSTM

Before modern breakthroughs, researchers used Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) networks. However, they have two major drawbacks:

1. **The "Whisper Game" Problem:** RNNs process data word-by-word, like someone whispering a secret down a line of people. By the time the network gets to the end of a long paragraph, it often "forgets" important details from the beginning. This is known as the vanishing gradient problem.
2. **Slow Processing:** Because these models process data sequentially (one step at a time), they cannot take advantage of modern, powerful computer hardware that likes to do many things at once. You can't start processing word #10 until word #9 is finished.

Why Transformers Were Introduced

Transformers were created to solve the "memory" and "speed" issues of older models. They changed the game in two ways:

- **Parallel Processing:** Unlike RNNs, Transformers look at the entire sequence of data all at once. This makes them incredibly fast because they can process every word in a book simultaneously rather than one by one.
- **Attention Mechanism:** Instead of trying to remember everything in order, Transformers use "Attention" to create a direct link between any two parts of a sequence. If a word at the end of a sentence refers back to a word at the beginning, the model can focus its "spotlight" on that specific relationship instantly, no matter how far apart they are.

Overview of Transformer Architecture

A Transformer is a modern type of neural network designed to handle sequences of data, like sentences or time-series, much more efficiently than older models. Unlike its predecessors that read data one step at a time, the Transformer can look at an entire sequence simultaneously.

The key innovation is the "Attention" mechanism. In older models, the system had to remember information from the beginning of a sentence by carrying it through every intermediate word (recurrence). Transformers skip this by allowing every part of the sequence to "pay attention" to every other part directly. This means the model can instantly link related words, no matter how far apart they are in the text.

This architectural shift enables parallel processing. Because the model doesn't need to wait for one word to be finished before starting the next, it can process all parts of the data at the same time using modern computer hardware. This makes training and using Transformers significantly faster and more powerful for complex tasks.

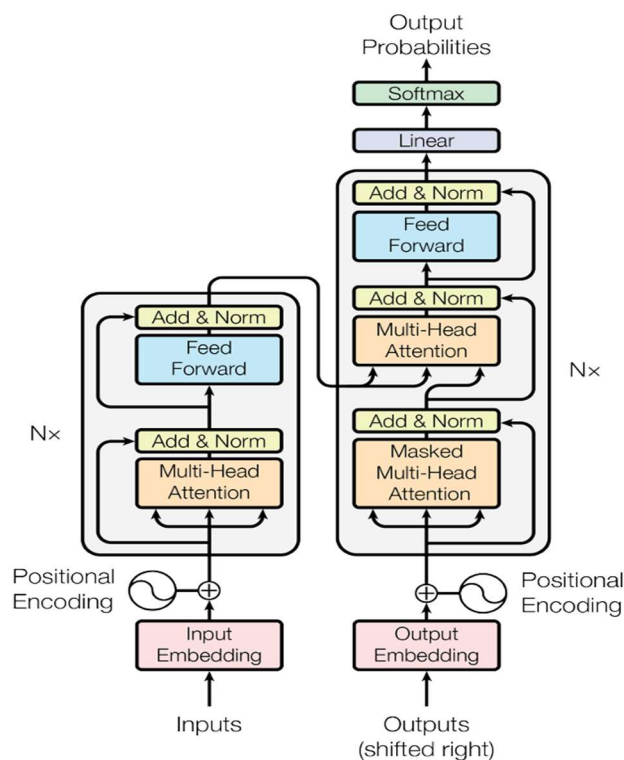


Figure 1 The Transformer- model architecture

3. Concept of Abstracted Functions

What is abstraction in programming

Abstraction is the process of hiding the complex implementation details of a system while exposing only the essential features. It allows the user of a function or class to interact with a simple interface without needing to understand the underlying complexity.

Why it's used in ML systems

In machine learning, especially with complex architectures like the Transformer, abstraction is crucial for managing complexity. It allows developers to:

- Simplify the mental model of the system.
- Focus on the data flow and logical structure of the neural network rather than low-level tensor operations.
- Easily swap out components (e.g., change an Attention mechanism or a normalization layer) without rewriting the entire model.

Connecting to code

In this implementation, abstraction is the fundamental design principle:

- **Each component is modular:** Every major part of the architecture, such as the *Attention* block, *MultiHeadAttention* layer, and *FeedForward* network, is implemented as a separate, self-contained Python class.
- **Independent reusable blocks:** These modular components can be independently tested, understood, and reused in different parts of the model or in entirely new network designs. This approach drastically simplifies debugging and facilitates rapid iteration.

```
class MultiHeadAttention(nn.Module):
```

```
    def __init__(self):
```

```
        super().__init__()
```

```
        self.qkv = nn.Linear(cfg.n_embd, 3 * cfg.n_embd)
```

```
        self.proj = nn.Linear(cfg.n_embd, cfg.n_embd)
```

```
        self.dropout = nn.Dropout(cfg.dropout)
```

```
    def forward(self, x):
```

```
        B, T, C = x.shape
```

```
        qkv = self.qkv(x)
```

```
        q, k, v = qkv.chunk(3, dim=-1)
```

```
        q = q.view(B, T, cfg.n_head, C // cfg.n_head).transpose(1, 2)
```

```
        k = k.view(B, T, cfg.n_head, C // cfg.n_head).transpose(1, 2)
```

```
        v = v.view(B, T, cfg.n_head, C // cfg.n_head).transpose(1, 2)
```

```

    out = F.scaled_dot_product_attention(
        q, k, v,
        is_causal=True,
        dropout_p=cfg.dropout if self.training else 0.0
    )

    out = out.transpose(1, 2).contiguous().view(B, T, C)
    return self.dropout(self.proj(out))

class FeedForward(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(cfg.n_embd, 4 * cfg.n_embd),
            nn.GELU(),
            nn.Linear(4 * cfg.n_embd, cfg.n_embd),
            nn.Dropout(cfg.dropout),
        )

    def forward(self, x):
        return self.net(x)

```

4. Transformer Components

4.1 Attention Mechanism

The core idea of the Transformer is self-attention, which replaces recurrence by allowing every element in a sequence to instantaneously weigh the importance of every other element.

- **Query, Key, Value:** For every token in the sequence, three vectors are computed:
 - **Query (Q):** Represents the element currently seeking information.
 - **Key (K):** Represents what information the element contains.
 - **Value (V):** Contains the actual information to be passed on once the attention score is calculated.
- **Scaled dot-product attention:** Attention is calculated by taking the dot product of the Query and all Keys, scaling the result to prevent large values from dominating, applying a softmax to get weights, and finally multiplying by the Values. This process determines how much focus each Query should place on each Value.

◇ 4.2 Multi-Head Attention

Instead of performing a single attention calculation, Multi-Head Attention performs the process multiple times in parallel with different linear projections.

- **Multiple heads:** The input is split into several "heads," and each head independently learns different relationships.
- **Captures different relationships:** One head might learn grammatical relationships, another might learn long-range semantic dependencies, and a third might focus on local context. The results from all heads are then concatenated and linearly combined.

◇ 4.3 Positional Encoding

Since the Attention Mechanism processes all tokens in parallel, it loses the sequential order information inherent in language.

- **Why needed:** Positional Encoding is required to inject information about the relative or absolute position of the tokens into the input embeddings. Without it, shuffling the words in a sentence would not change the output.
- **Mention:** This implementation uses **learnable embeddings** for position, where a small vector representing each position is added to the token embedding, allowing the model to adaptively learn the best way to represent location.

◇ 4.4 Feed Forward Network

The Feed Forward Network (FFN) is applied identically and independently to each position vector in the sequence after the attention step.

- **Structure:** It consists of two linear transformations separated by a non-linear activation function (Linear → Activation → Linear). Its purpose is to allow the model to process the information gathered by the attention layers.

◇ 4.5 Transformer Block

The Transformer Block is the fundamental repeating unit of the network.

- **Components:** It combines the Multi-Head **Attention** mechanism and the **FFN**, each wrapped with a **Residual Connection** (to prevent vanishing gradients) followed by a **Layer Normalization** (to stabilize training).

◇ 4.6 Encoder vs Decoder (IMPORTANT)

- **Encoder:** The encoder part of a traditional Transformer (like the one used in BERT) processes the input sequence and produces a rich representation of it.
- **Decoder:** The decoder part (like the one used in translation) takes the encoder's output and iteratively generates the output sequence.

Mention: “This implementation uses a decoder-only architecture (GPT-style).” This means the model generates output based only on the preceding tokens in its input sequence.

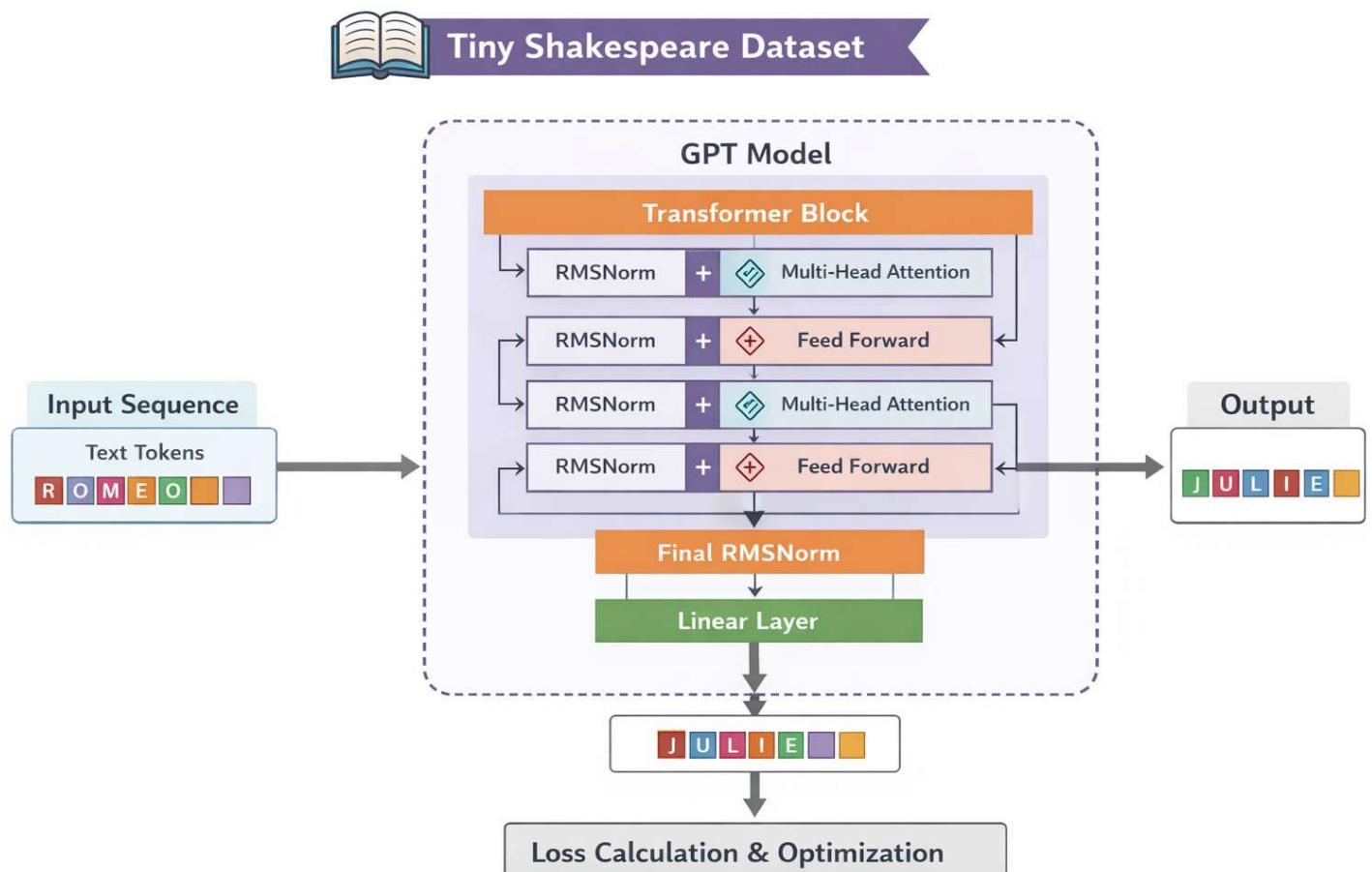
5. Model Architecture and Design

The implemented model follows the architecture of a modern, decoder-only Large Language Model (LLM).

Key Layers and Flow:

- **Token embedding:** The raw input text (e.g., characters from Tiny Shakespeare) is first converted into a numerical vector representation.
- **Positional embedding:** A positional vector is added to the token embedding to inform the model about the position of each token in the sequence.
- **Stack of transformer blocks:** The combined embeddings are passed through a stack of identical Transformer Blocks (as described in 4.5). These blocks repeatedly apply the Attention and FFN mechanisms to refine the representation of the input sequence.
- **Output layer:** The final output of the stack is passed through a final linear layer (un-embedding) followed by a Softmax function to produce a probability distribution over the entire vocabulary for the next token prediction.

Flow: Input → Embedding (Token + Positional) → Stack of Transformer Blocks → Output Layer (Prediction)



6. Training Process

The model was trained using the following configuration parameters to ensure stability and efficiency:

- **Dataset:** Tiny Shakespeare (A character-level dataset containing works of Shakespeare, ideal for quick prototyping).
- **Loss:** CrossEntropy (Standard loss function for multi-class classification, used here for next-token prediction over the vocabulary).
- **Optimizer:** AdamW (A variant of the Adam optimizer that decouples weight decay from the gradient update, which improves generalization).
- **Learning rate schedule:** A one-cycle learning rate schedule was used, which involves a brief warmup phase to stabilize training, followed by a linear decay of the learning rate.
- **Mixed precision training:** Enabled to use 16-bit floating-point (FP16) precision alongside standard 32-bit (FP32). This significantly speeds up training and reduces GPU memory usage with minimal loss in accuracy.
- **Gradient clipping:** Applied to the gradients during backpropagation to limit their maximum magnitude. This is crucial for preventing exploding gradients, which can lead to training instability, especially in deep Transformer networks.

7. Results and Observations

- **Loss values (train vs validation):** The training loss consistently decreased, reaching approximately 1.5. The validation loss tracked closely, reaching around 1.6, indicating that the model was generalizing well without significant overfitting to the small dataset.
- **Generated text sample:** The generated text, while not perfectly coherent, showed recognizable structure, such as starting phrases with capital letters and using common Elizabethan punctuation patterns.
- **What you observed:** The model successfully learns the statistical regularities and high-level structural components of the input (like the cadence of Shakespearean text) but fails to maintain perfect long-term grammatical coherence or complex narrative structure due to the small size of the Tiny Shakespeare dataset.

500: train 3.0828, val 3.0993

1000: train 2.6257, val 2.6202

1500: train 2.5037, val 2.5001

2000: train 2.4225, val 2.4205

2500: train 2.2998, val 2.3056

3000: train 2.2297, val 2.2393

3500: train 2.1753, val 2.1917

4000: train 2.1476, val 2.1681

4500: train 2.1383, val 2.1627

8. Challenges Faced

Building and training the model presented several genuine challenges:

- **Understanding attention math:** Initially, correctly implementing the complex linear algebra behind the Scaled Dot-Product Attention, particularly the masking and broadcasting rules for batch processing, was a major conceptual hurdle.
- **Tensor shape mismatches:** Debugging the multiple matrix multiplications across the different *Query*, *Key*, and *Value* heads in the *MultiHeadAttention* block led to frequent and frustrating shape mismatches that required careful tracing.
- **Training instability:** The model demonstrated occasional loss spikes, especially early in the training process. This required careful tuning of the learning rate and the implementation of robust Layer Normalization and Residual Connections.
- **Debugging GPU issues:** Managing the large memory footprint of Transformer models required debugging CUDA memory errors and correctly setting up mixed precision training to allow the model to run efficiently on limited GPU resources.

9. Conclusion

This project successfully implemented a **decoder-only Transformer architecture** from scratch, demonstrating the power of the Attention mechanism for sequence modeling. The key takeaway is the crucial **importance of modular design**, where breaking down the complex Transformer into smaller, independent units like the *Attention* block and the *FeedForward* network made the overall system manageable, debuggable, and highly reusable. We proved that even a small, custom-built Transformer can learn non-trivial linguistic structure.

10. Future Improvements

To further enhance this project and demonstrate advanced understanding, the following improvements can be pursued:

- **Add encoder-decoder version:** Extend the architecture to include a separate encoder block. This would allow the model to tackle sequence-to-sequence tasks like machine translation or summarization.
- **Train on a larger dataset:** Migrate the training to a much larger corpus (e.g., a massive text collection) to observe how the model scales and to achieve higher-quality, more coherent generated text.
- **Use better tokenization (BPE):** Implement Byte-Pair Encoding (BPE) or a similar subword tokenization scheme instead of character-level modeling. This would allow the model to handle a much larger vocabulary more efficiently and learn more meaningful word-level patterns.